

Apostila de Linguagem C

Silas Reuel da Silva

Informática 1º Ano

Conteúdo: Informações gerais de programação em C

Novembro de 2023

Nome: _____

Turma: _____

Bauru, 2023

Sumário

Apostila de Linguagem C	1
Menus em C	3
Vetores - Array	4
Matrizes	6
Funções	8
Manipulação de Strings	12
Teclas de Atalho do Dev C++	17
Teclas de Atalho do SO (Sistema Operacional)	17
Biblioteca Windows.h	18
Struct	19
REFERÊNCIAS	20

Menus em C

Para montar um menu de seleção em C utilizamos 'switch case'.

Todo menu deve aparecer sempre que alguma operação for finalizada, exceto se receber um comando para parada. Para realizarmos isso usamos 'do while'.

No 'do while' ele faz uma operação e verifica a condição depois.

Exemplos:

Menu com switch case com inteiros:

```
#include<stdio.h>
main(){
    int Variavel_Selecao; //Declaração da variável
    do{
        printf("Escolha: 1)Fazer algo; 2)Parar o programa\n");
        scanf("%d", &Variavel_Selecao); //lê a variável
        switch(Variavel_Selecao){ //Variável para fazer 'funfar' o switch
            case 1:
                printf("Faz algo\n");
                break;
            case 2:
                printf("Para o programa\n");
                break; //Para o switch
        }
    }while(Variavel_Selecao != 2); //quando for igual a 2 para o código
}
```

*Funfar: Funcionar

Menu com switch case com caracteres:

```
#include<stdio.h>
#include<conio.h> //biblioteca para manipular char
main(){
    char Variavel_Selecao;
    do{
        printf("Escolha: A)Fazer algo; b)Parar o programa\n");
        fflush(stdin); // limpa o buffer de teclado, deve vir sempre antes da função de char e string
        Variavel_Selecao = getchar(); // É passado o valor para a variável através do comando getchar()
        switch(Variavel_Selecao){ //Variável para fazer 'funfar' o switch
            case 'A':
                printf("Faz algo\n");
                break;
            case 'b':
                printf("Para o programa\n");
                break; //Para o switch
        }
    }while(Variavel_Selecao != 'b'); //quando for igual a b para o código
}
```

Vetores - Array

O vetor é uma estrutura de dados indexada, que pode armazenar uma determinada quantidade de valores do mesmo tipo.

Exemplo:

```
#include<stdio.h>
main(){
    int vetor[2]; // declarado um vetor de 2 linhas
    int a=0; // Variável criada para deixar o processo automático, sem input de usuário
    for(int l = 0; l < 2; l++){ // l é a variável para atualizar a linha
        a += 5; // Soma 5 na variável é o mesmo de fazer a = a + 5
        vetor[l] = a; // adiciona o valor na posição variável(l)
    }
    for(int l = 0; l < 2; l++){ // l é a variável para atualizar a linha
        printf("%d\n", vetor[l]); // Imprime o vetor
    }
}
```

Outro vetor (array) muito utilizado são as strings, que são arrays de char, exemplo:

```
#include<stdio.h>
#include<string.h> // Biblioteca para manipular strings
#include<locale.h> // Biblioteca para usar o setlocale
main(){
    char string[20]; // declarando uma string de 20 caracteres
    printf("Escreva uma frase: ");
    fflush(stdin); // Limpa o buffer de teclado, assim não vai lixo
    gets(string); // pega o valor da string através de input de usuário
    puts(string); // Imprime a string
    // setlocale está aqui pois ele muda o que você coloca no input de usuário
    setlocale(LC_ALL, "portuguese"); // Muda para o padrão da língua portuguesa
    char stringPreMontada[] = {'O', 'l', 'á', '!', ' '}; // string com valor pré-montada
    // Os colchetes ficam vazios
    puts(stringPreMontada); // Imprime a string pré-montada
}
```

Impressão no terminal (cmd/prompt de comando):

```
Escreva uma frase: café é bom
café é bom
Olá!
```

Vetor de inteiros e floats pré-montados:

```
#include<stdio.h>
main(){
    int inteiros[] = {0, 1, 2, 3}; // declarando um vetor de inteiros pré-montado
    float NumsComVirgula[] = {0, 0.1, 5.6}; // declarando um vetor de flutuantes pré-montado
    for(int l = 0; l < 4; l++){ // laço for para atualizar a linha
        printf("%d\n", inteiros[l]); // Imprime o vetor de inteiros
    }
    for(int l = 0; l < 3; l++){ // laço for para atualizar a linha
        printf("%.2f\n", NumsComVirgula[l]); // Imprime o vetor de flutuantes
    }
}
```

Impressão no terminal (cmd/prompt de comando):

```
0
1
2
3
0.00
0.10
5.60
```

Matrizes

Construir matriz com vários laços for:

```
#include<stdio.h>
main(){
    int matriz[2][2]; //Criação da matriz com 2 linhas e 2 colunas
    for(int l = 0; l < 2; l++){ //l é a variável para atualizar a linha
        for(int c = 0; c < 2; c++){ //laço for para atualizar a coluna na posição 1
            matriz[l][0] = 1; //Definimos o valor da matriz na linha variável
            //e na coluna 1
        }
        for(int c = 1; c < 2; c++){ //laço for para atualizar a coluna na posição 2
            matriz[l][1] = 2; //Definimos o valor da matriz na linha variável
            //e na coluna 2
        }
    }
    for(int l = 0; l < 2; l++){ //l é a variável para atualizar a linha
        for(int c = 0; c < 2; c++){ //c é a variável para atualizar a coluna
            printf("%d\t", matriz[l][c]); //Imprime a matriz
            //\t serve para imprimir com espaçamento entre colunas
        }
        printf("\n"); // \n serve para imprimir com espaçamento entre linhas
    }
}
```

Impressão no terminal (cmd/prompt de comando):

```
1      2
1      2

-----
Process exited after 1.175 seconds with return value 0
Pressione qualquer tecla para continuar. . .
```

Observe o tempo de execução do código feito com vários laços for: 1,175 segundos, não parece muita coisa, porém tivemos esse resultado pois eram poucas linhas e colunas, e se fossem mais?

Construir matriz com if ao invés de vários laços for:

```
#include<stdio.h>
main(){
    int matriz[2][2]; //Criação da matriz com 2 linhas e 2 colunas
    for(int l = 0; l < 2; l++){ //l é a variável para atualizar a linha
        for(int c = 0; c < 2; c++){ //c é a variável para atualizar a coluna
            if(c==0){ //condição que verifica se estamos na coluna 1
                matriz[l][0] = 1; //Definimos o valor da matriz na linha variável
                //e na coluna 1
            }
            else if(c==1){ //condição que verifica se estamos na coluna 2
                matriz[l][1] = 2; //Definimos o valor da matriz na linha variável
                //e na coluna 2
            }
        }
    }
    for(int l = 0; l < 2; l++){ //l é a variável para atualizar a linha
        for(int c = 0; c < 2; c++){ //c é a variável para atualizar a coluna
            printf("%d\t", matriz[l][c]); //Imprime a matriz
            //\t serve para imprimir com espaçamento entre colunas
        }
        printf("\n"); // \n serve para imprimir com espaçamento entre linhas
    }
}
```

Impressão no terminal (cmd/prompt de comando):

```
1      2
1      2

-----
Process exited after 0.6503 seconds with return value 0
Pressione qualquer tecla para continuar. . .
```

Observe o tempo de execução do código feito com ifs ao invés de vários laços for: 0,6503 segundos, não deu 1 segundo para que fosse terminada a execução do programa.

Essas duas formas acima são para casos que tenhamos que construir uma tabela com valores específicos

Funções

Função com passagem por valor, exemplos:

Tipo VOID, não retorna valor:

```
#include<stdio.h>
void funcao(int N1, int N2){ //criação da função, não retorna valor (função tipo void)
    int resultado; //Declaração de variável
    resultado = N1 + N2; //Operação matemática de soma
    printf("\nResultado = %d", resultado); //Imprime o resultado
}

main(){
    int N1, N2; //Declaração da variável
    printf("Digite dois valores: ");
    scanf("%d %d", &N1, &N2); // Outra forma de fazer a leitura, pega dois valores de uma vez
    funcao(N1, N2); // chamamento da função, passando como parâmetro as variáveis
    funcao(4, 3); // Passagem de parâmetro utilizando números pré-determinados
}
```

Tipo INT, retorna valor:

```
#include<stdio.h>
int funcao(int N1, int N2){ //criação da função, retorna valor (função tipo int)
    int resultado; //Declaração de variável
    resultado = N1 + N2; //Operação matemática de soma
    return resultado; //Retorna a variável resultado para o main
}

main(){
    int N1, N2, result; //Declaração das variáveis
    printf("Digite dois valores: ");
    scanf("%d %d", &N1, &N2); // Outra forma de fazer a leitura, pega dois valores de uma vez
    result = funcao(N1, N2); // chamamento da função, passando como parâmetro as variáveis
    //A variável result está recebendo o valor do retorno
    printf("\nResultado: %d\n", result);
    result = funcao(4, 3); // Passagem de parâmetro utilizando números pré-determinados
    //A variável result está recebendo o valor do retorno
    printf("Resultado: %d", result); //Imprime o valor adquirido na função
}
```


Tipo FLOAT, retorna valor:

```
#include<stdio.h>
float funcao(float N1, float N2){ //criação da função, retorna valor (função tipo float)
    float resultado; //Declaração de variável
    resultado = N1 + N2; //Operação matemática de soma
    return resultado; //Retorna a variável resultado para o main
}

main(){
    float N1, N2, result; //Declaração das variáveis
    printf("Digite dois valores: ");
    scanf("%f %", &N1, &N2); // Outra forma de fazer a leitura, pega dois valores de uma vez
    result = funcao(N1, N2); // chamamento da função, passando como parâmetro as variáveis
    //A variável result está recebendo o valor do retorno
    printf("\nResultado: %f\n", result);
    result = funcao(4, 3); // Passagem de parâmetro utilizando números pré-determinados
    //A variável result está recebendo o valor do retorno
    printf("Resultado: %f", result); //Imprime o valor adquirido na função
}
```

Função recebendo como parâmetro uma matriz:

```
#include<stdio.h>
void funcao(int matriz[][2]){
    for(int l = 0; l < 2; l++){ //l é a variável para atualizar a linha
        for(int c = 0; c < 2; c++){ //c é a variável para atualizar a coluna
            if(c==0){ //condição que verifica se estamos na coluna 1
                matriz[l][0] = 1; //Definimos o valor da matriz na linha variável
                //e na coluna 1
            }
            else if(c==1){ //condição que verifica se estamos na coluna 2
                matriz[l][1] = 2; //Definimos o valor da matriz na linha variável
                //e na coluna 2
            }
        }
    }
}

main(){
    int matriz[2][2];
    funcao(matriz);
    for(int l = 0; l < 2; l++){ //l é a variável para atualizar a linha
        for(int c = 0; c < 2; c++){ //c é a variável para atualizar a coluna
            printf("%d\t", matriz[l][c]); //Imprime a matriz
            //\t serve para imprimir com espaçamento entre colunas
        }
        printf("\n"); // \n serve para imprimir com espaçamento entre linhas
    }
}
```

Impressão no terminal (cmd/prompt de comando):

1	2
1	2

Funções com Passagem por Referência

Funções com passagem por referência são importantes pois as alterações feitas dentro da função têm efeito direto nas variáveis passadas como argumentos na chamada.

Exemplos:

Passagem de variável para função por referência:

```
#include<stdio.h>
void funcao(int *a){ //declaramos a variável com um '*', chamamos isso de ponteiro
    *a = *a + 10; // Neste momento a variável *a passa a ser 12, e a que está em main também
}

main(){
    int a=2; // A variável passada como parâmetro não é declarada com ponteiro
    funcao(&a); // Ao chamar a função, a variável passada como parâmetro é passada com '&'
    printf("Valor atualizado: %d", a); // A saída na impressão é 12, note que não usamos return
}
```

OBSERVAÇÃO: A variável não necessita ser igual em main e na função.

Passar uma matriz por referência, exemplo:

```

#include<stdio.h>
void funcao(int matriz[][2]){
    for(int l = 0; l < 2; l++){ //l é a variável para atualizar a linha
        for(int c = 0; c < 2; c++){ //c é a variável para atualizar a coluna
            if(c==0){//condição que verifica se estamos na coluna 1
                matriz[l][0] = 1;//Definimos o valor da matriz na linha variável
                //e na coluna 1
            }
            else if(c==1){//condição que verifica se estamos na coluna 2
                matriz[l][1] = 2;//Definimos o valor da matriz na linha variável
                //e na coluna 2
            }
        }
    }
}

main(){
    int matriz[2][2];
    funcao(matriz);
    for(int l = 0; l < 2; l++){ //l é a variável para atualizar a linha
        for(int c = 0; c < 2; c++){ //c é a variável para atualizar a coluna
            printf("%d\t", matriz[l][c]); //Imprime a matriz
            //\t serve para imprimir com espaçamento entre colunas
        }
        printf("\n");// \n serve para imprimir com espaçamento entre linhas
    }
}

```

Observe que é o mesmo exemplo da passagem por parâmetro, automaticamente são mudados os valores na passagem, a matriz foi construída na função void e impressa em main, pode-se dizer que a passagem de uma matriz por parâmetro é a mesma que a passagem por referência, o mesmo vale para vetores.

Passagem de vetores para função:

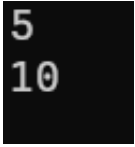
```

#include<stdio.h>
void funcao(int vetor[]){// Na função recebemos um vetor, os colchetes são vazios
    int a=0;//Variável criada para deixar o processo automático, sem input de usuário
    for(int l = 0; l < 2; l++){ //l é a variável para atualizar a linha
        a += 5;// Soma 5 na variável é o mesmo de fazer a = a + 5
        vetor[l] = a;// adiciona o valor na posição variável(l)
    }
}

main(){
    int vetor[2];// declarado um vetor de 2 linhas
    funcao(vetor);// Passamos como parâmetro apenas o nome do vetor
    for(int l = 0; l < 2; l++){ //l é a variável para atualizar a linha
        printf("%d\n", vetor[l]); //Imprime o vetor
    }
}

```

Impressão no terminal (cmd/prompt de comando):



```
5
10
```

Manipulação de Strings

gets e puts:

```
#include<stdio.h>
#include<string.h> // Biblioteca para manipular strings
main(){
    char string1[20]; // Definida variável char de 20 caracteres (string)

    printf("Escreva uma frase curta: ");
    fflush(stdin); // Limpa buffer de teclado
    gets(string1); // Lê a variável
    puts(string1); // Imprime a string
```

Impressão no terminal (cmd/prompt de comando):

```
Escreva uma frase curta: Olá Mundo!
Olá Mundo!
```

strcat:

```
#include<stdio.h>
#include<string.h> // Biblioteca para manipular strings
main(){
    char string1[20]; // Definida variável char de 20 caracteres (string)
    char string2[20];

    printf("Escreva uma frase curta: ");
    fflush(stdin); // Limpa buffer de teclado
    gets(string1); // Lê a variável

    printf("\nEscreva uma frase curta: ");
    fflush(stdin);
    gets(string2);

    strcat(string1, string2);
    puts(string1); // concatenar strings, cat=concatenar
    //sintaxe: strcat(string1, string2);
    //nesse caso a string1 passa a ser seu valor e ao final o valor da string 2
    //Observe que estamos imprimindo o valor contido na string1
```

Impressão no terminal (cmd/prompt de comando):

```
Escreva uma frase curta: Olá
Escreva uma frase curta: Mundo
OláMundo
```

strlen:

```
#include<stdio.h>
#include<string.h> // Biblioteca para manipular strings
main(){
    char string1[20]; // Definida variável char de 20 caracteres (string)

    printf("Escreva uma frase curta: ");
    fflush(stdin); // Limpa buffer de teclado
    gets(string1); // Lê a variável

    int tamanho = strlen(string1); // tamanho da string, len=length=comprimento
    // vale ressaltar que também são contados os espaços
    printf("Tamanho da string: %d\n", tamanho);
}
```

Impressão no terminal (cmd/prompt de comando):

```
Escreva uma frase curta: Olá Mundo!
Tamanho da string: 10
```

strcpy:

```
#include<stdio.h>
#include<string.h> // Biblioteca para manipular strings
main(){
    char string1[20]; // Definida variável char de 20 caracteres (string)
    char string2[20];

    printf("Escreva uma frase curta: ");
    fflush(stdin); // Limpa buffer de teclado
    gets(string1); // Lê a variável

    printf("\nEscreva uma frase curta: ");
    fflush(stdin);
    gets(string2);

    strcpy(string2, string1); // copiar string, cpy=copy
    puts(string2); // printa a string2 que agora tem o valor da string1
    // sintaxe: strcpy(string_destino, string_original);
}
```

Impressão no terminal (cmd/prompt de comando):

```
Escreva uma frase curta: Olá
Escreva uma frase curta: Mundo!
Olá
```

strcmp:

Compara duas variáveis, se forem iguais, retorna 0, se não, retorna 1 ou -1.

```
#include<stdio.h>
#include<string.h> // Biblioteca para manipular strings
main(){
    char string1[20]; // Definida variável char de 20 caracteres (string)
    char string2[20];

    printf("Escreva uma frase curta: ");
    fflush(stdin); // Limpa buffer de teclado
    gets(string1); // Lê a variável

    printf("\nEscreva uma frase curta: ");
    fflush(stdin);
    gets(string2);

    // comparar strings, cmp=comparar
    // sintaxe: strcmp(string1, string2); Compara duas strings
    if(strcmp(string2, string1)){ // Se for igual à 1 ou -1
        printf("\nDiferentes");
    }
    else{ // Se não for igual a 1 ou -1
        printf("\nIguais");
    }
}
```

Impressões no terminal (cmd/prompt de comando):

```
Escreva uma frase curta: Olá
Escreva uma frase curta: Mundo
Diferentes
```

```
Escreva uma frase curta: Olá
Escreva uma frase curta: Olá
Iguais
```

OBSERVAÇÃO: É case sensitive, ou seja, diferencia maiúsculas de minúsculas.

Mesmo que seja a mesma frase, se estiver escrito em um com maiúsculas e no outro com minúsculas, irá retornar que as strings são diferentes.

strcmpi:

```
#include<stdio.h>
#include<string.h> // Biblioteca para manipular strings
main(){
    char string1[20]; // Definida variável char de 20 caracteres (string)
    char string2[20];

    printf("Escreva uma frase curta: ");
    fflush(stdin); // Limpa buffer de teclado
    gets(string1); // Lê a variável

    printf("\nEscreva uma frase curta: ");
    fflush(stdin);
    gets(string2);

    // comparar strings, cmp=comparar, i=ignorar
    // sintaxe: strcmpi(string1, string2); Compara duas strings ignora case sensitive
    if(strcmpi(string2, string1)){ // Se for igual à 1 ou -1
        printf("\nDiferentes");
    }
    else{ // Se não for igual a 1 ou -1
        printf("\nIguais");
    }
}
```

Impressões no terminal (cmd/prompt de comando):

```
Escreva uma frase curta: Olá
Escreva uma frase curta: olá
Iguais
```

OBSERVAÇÃO: Ignora maiúsculas e minúsculas

strstr:

Retorna TRUE quando houver ocorrência de uma frase em outra.

```
#include<stdio.h>
#include<string.h> // Biblioteca para manipular strings
main(){
    char string1[20]; // Definida variável char de 20 caracteres (string)
    char string2[20];

    printf("Escreva uma frase curta: ");
    fflush(stdin); // Limpa buffer de teclado
    gets(string1); // Lê a variável

    printf("\nEscreva uma frase curta: ");
    fflush(stdin);
    gets(string2);

    // sintaxe: strstr(string1, string2);
    if(strstr(string1, string2)){ // Verifica a ocorrência de uma frase em outra
        printf("Frase encontrada");
    }
    else{
        printf("Frase nao encontrada");
    }
}
```

Impressões no terminal (cmd/prompt de comando):

```
Escreva uma frase curta: Olá Mundo
Escreva uma frase curta: Café
Frase nao encontrada
```

```
Escreva uma frase curta: Olá Mundo
Escreva uma frase curta: Mundo
Frase encontrada
```

OBSERVAÇÃO: Também é case sensitive.

strupr e strlwr:

```
#include<stdio.h>
#include<string.h> // Biblioteca para manipular strings
main(){
    char string1[20]; // Definida variável char de 20 caracteres (string)

    printf("Escreva uma frase curta: ");
    fflush(stdin); // Limpa buffer de teclado
    gets(string1); // Lê a variável

    // sintaxe: strupr(string1);
    // sintaxe: strlwr(string1);
    puts(strupr(string1)); // Aumentar a letra; upr=uppercase=Maiúscula
    puts(strlwr(string1)); // Diminuir a letra; lwr=lowercase=Minúscula
}
```

Impressão no terminal (cmd/prompt de comando):

```
Escreva uma frase curta: Aprendi Tudo
APRENDI TUDO
aprendi tudo
```

DICA: upr=Upper; lwr=lower

Teclas de Atalho do Dev C++

Ctrl+D = Deletar a linha na qual o cursor está

Ctrl+E = Duplica a linha na qual o cursor está

Ctrl+S = Salva o programa

Ctrl+, = Descomenta a linha na qual o cursor está

Ctrl+. = Comenta a linha na qual o cursor está

Ctrl+; = Comenta e descomenta a linha na qual o cursor está

Ctrl+Shift+Seta para cima = Move a linha em que o cursor está para cima

Ctrl+Shift+Seta para baixo = Move a linha em que o cursor está para baixo

Teclas de Atalho do SO (Sistema Operacional)

Ctrl+Seta para esquerda = Percorre de palavra em palavra da direita para esquerda

Ctrl+Seta para direita = Percorre de palavra em palavra da esquerda para direita

Biblioteca Windows.h

É uma biblioteca que tem acesso direto ao prompt de comando, podendo substituir a biblioteca conio.c que provavelmente você não vai ter 😊

```
#include<stdio.h>
#include<Windows.h> // Biblioteca que substitui a conio.c
//para usar os comandos dessa biblioteca é necessário saber um pouco de cmd
main(){
    system("color help");//Acessa o cmd e por trás roda o comando mostra uma tabela de comandos para mudar a cor da letra
    system("color a");//Acessa o cmd e por trás roda o comando muda a cor da letra para verde
    Sleep(3000);//Dá um delay de 3 segundos (1000= 1 segundo) é igual o delay do arduino
    system("cls");//Acessa o cmd e por trás roda o comando que limpa o cmd
    system("explorer");//Acessa o cmd e por trás roda o comando abre o explorador de arquivos
    exit(0);//Para o código que está rodando
}
```

Atenção!! Não dê comando de Task kill caso não saiba o que está fazendo, isso pode ocasionar sérios problemas na sua máquina.

Tente pesquisar comandos do cmd, dessa forma poderá compreender melhor como funciona seu Sistema Operacional.

Struct

Structs armazenam vários tipos de dados.

```
#include <stdio.h>

typedef struct //cria uma struct
{
    char modelo[30]; //string
    float potenciaMotor; //números com vírgula
    int anoFabricacao, //inteiros
        numPortas; //faz parte do int
}CARRO; //key(chave)

void Exibe(CARRO car) //função que recebe a struct
{
    printf("\n\tExibindo carro\n");
    printf("Modelo: %s\n", car.modelo); //imprime os valores
    printf("Motor: %.1f\n", car.potenciaMotor);
    printf("Ano: %d\n", car.anoFabricacao);
    printf("%d portas\n", car.numPortas);
}

void Preenche(CARRO *car) //recebe a função por referência
{
    printf("Modelo do carro: ");
    gets( car->modelo ); //coleta os dados

    printf("Motor: ");
    scanf("%f", &car->potenciaMotor);

    printf("Ano: ");
    scanf("%d", &car->anoFabricacao);

    printf("Numero de portas: ");
    scanf("%d", &car->numPortas);
}

int main(void)
{
    CARRO fusca; //struct
    Preenche(&fusca); //envio para função por referência
    Exibe(fusca); //envio para função por valor

    return 0; //Encerra o código
}
```

Termina aqui a apostila de C, com algumas coisas importantíssimas e que são bem difíceis de compreender quando não se tem uma boa explicação, eu fico por aqui, bons estudos para você e bom desenvolvimento em C. Grande abraço!

REFERÊNCIAS

Passagem por Referência - Como copiar vetor e matriz em C. Disponível em: https://www.cprogressivo.net/2013/03/Como-copiar-um-vetor-ou-matriz-em-C_7.html. Acesso em: 16 nov. 2023.

Disponível em: <https://linguagemc.com.br/funcao-com-passagem-por-referencia/>. Acesso em: 16 nov. 2023.

como passar uma matriz por referência para uma função [C / C++].

Disponível em: <https://www.vivaolinux.com.br/topico/C-C++/como-passar-uma-matriz-por-referencia-para-uma-funcao>. Acesso em: 16 nov. 2023.

Passagem de matriz por referência. Disponível em: https://www.clubedohardware.com.br/forums/topic/849249-passagem-de-matriz-por-refer%C3%Aancia/#google_vignette. Acesso em: 16 nov. 2023.